

Sistemas-de-Archivos

¿Qué es un file descriptor? Nombrar 3 system calls que los afecten.

Es un identificador para un archivo o un recurso de entrada/salida como puede ser un pipe o un socket. Estos son específicos al proceso que los crea.

- `open("file.txt", 0_RDONLY)`: Abre un archivo y devuelve un file descriptor.
- `read(fd, buffer, size)`: Lee datos de un file descriptor.
- `close(fd)`: Cierra un file descriptor.
- `dup2(fd, 1)`: dado el file descriptor *fd* que hace referencia a un archivo, podemos usar este wrapper de syscall para hacer que el stdout de nuestro programa vaya al archivo.

¿Cuándo se revisan los permisos de acceso sobre un archivo? Explicar por qué el file descriptor se crea cuando hacemos un open y no se vuelven a revisar los permisos.

Se revisan al momento de abrir el archivo. El proceso considera que el archivo fue autorizado para acceder al archivo.

¿Qué es un FS y para qué sirve?

Un file system es un esquema jerárquico para organizar archivos de un sistema. De esta manera podemos organizar los bytes en nombres y directorios para tener un mejor orden.

¿Cuándo es adecuado reservar espacio en disco de manera secuencial? ¿Qué beneficios nos trae? (CD-ROM, ISO-9660)

¿Cuál FS nos conviene utilizar para un sistema embebido: FAT o inodos?

FAT porque es más rápido y sencillo, inodos es más útil para sistemas complejos donde la modificación es constante y es necesaria una estructura más rigurosa.

¿Cuál FS nos conviene utilizar para implementar UNDO (deshacer la última escritura a disco)? ¿Cómo se implementaría en FAT y en inodos?

Nos convienen inodos porque tienen mejor soporte para journaling. Facilita el registro de operaciones y la revisión de estados previos.

Fat puede ser modificado pero implicaría un impacto significativo en la eficiencia. Deberíamos copiar los bloques afectados con su index o la tabla entera para cada cambio en disco. Esto implica MUCHO desperdicio de espacio.

¿Cuál FS nos conviene utilizar para implementar un backup total? ¿Cómo se implementaría en FAT y en inodos?

Para fat hay que empezar desde la raíz leyendo todos los archivos/subdirectorios, extraer metadatos, nombres, atributos y ubicacion inicial de cada archivo y ademas copiarlos en orden en una nueva tabla fat. Es simple pero carece de estructuras eficientes para grandes volumenes de datos.

Para ext2 recorremos el sistema de archivos de a inodos, en un sentido conceptual es mas simple de ver. Ademas de que los metadatos estan mas estructurados y facilita su copia con los permisos/atributos.

Para cada inodo viejo, creamos un inodo nuevo, copiamos sus metadatos y luego, de manera recursiva, copiamos los bloques. Es una forma mas estructurada de manejarse.

¿Cuál FS nos conviene utilizar para implementar un backup incremental? ¿Cómo se implementaría en FAT y en inodos?

¿Cuál FS nos conviene utilizar para implementar snapshot? (Diferenciar el caso en que queramos tomar una snapshot mientras el sistema está corriendo) ¿Cómo se implementaría en FAT y en inodos?

Explicar las diferencias entre FAT e inodos. Ventajas y desventajas de cada uno.

FAT se basa en tener una tabla entera en memoria que nos dice, para un archivo, donde comienza su primer bloque y donde es el segundo, asi sucesivamente hasta llegar a un EOF. Por cada bloque me dice donde esta el siguiente bloque.

Inodos se basa en tener un nodo por archivo donde tenemos sus metadatos como modificacion, tamaño, permisos etc. Es bastante eficiente y organizado porque permite tener los bloques del archivo en el mismo inodo (si el archivo es pequeño podemos usar las direcciones directas para leerlo). Ademas en inodos tenemos la ventaja de que hay permisos (aumenta la seguridad, distincion entre root y user), el nombre no tiene limite y los inodos se cargan a medida que se necesitan a memoria.

¿FAT implementa algún tipo de seguridad?

FAT no maneja seguridad. Hay características de archivos como readonly o hidden pero no existe el concepto de user vs kernel. Los archivos son iguales para cualquier grupo.

Explicar qué es journaling.

El sistema de archivos ext3 admite una característica popular llamada registro en journal, por la cual las modificaciones al sistema de archivos se escriben secuencialmente en un journal.

Un conjunto de operaciones que realizan una tarea específica es una transaction.

Una vez que una transaction se escribe en el journal, se considera confirmada.

Mientras tanto, las entradas del journal relacionadas con la transaction se reproducen en las estructuras reales del sistema de archivos.

A medida que se realizan los cambios, se actualiza un puntero para indicar qué acciones se han completado y cuáles aún están incompletas.

Cuando se completa una transaction confirmada completa, se elimina del journal. El journal, que en realidad es un búfer circular, puede estar en una sección separada del sistema de archivos, o incluso puede estar en un eje de disco separado. Es más eficiente, pero más complejo, tenerlo bajo cabezales de lectura y escritura separados, lo que reduce la contención de cabezales y los tiempos de búsqueda.

Si el sistema falla, algunas transactiones pueden permanecer en el journal. Esas transactiones nunca se completaron en el sistema de archivos a pesar de que fueron confirmadas por el sistema operativo, por lo que deben completarse una vez que el sistema se recupere.

Describir ext2.

Me lo se de memoria

¿Qué mantiene un inodo? ¿Cómo es la estructura de directorios?

Un inodo mantiene un conjunto de metadatos y una "lista" de direcciones directas (LBAs), una LBA llamada direcciones indirectas que apunta a una tabla de LBAs, una LBA llamada direcciones indirectas dobles que apunta a una tabla de tablas de LBAs y por último una LBA llamada direcciones indirectas triples que apunta a una tabla de tablas que apuntan a tablas de LBAs.

La estructura de directorios es también un inodo pero los bloques de datos en vez de ser datos de un archivo se tienen que parsear como direntries donde las entradas son otros archivos o subdirectorios. Esta se compone por el número de inodo en el

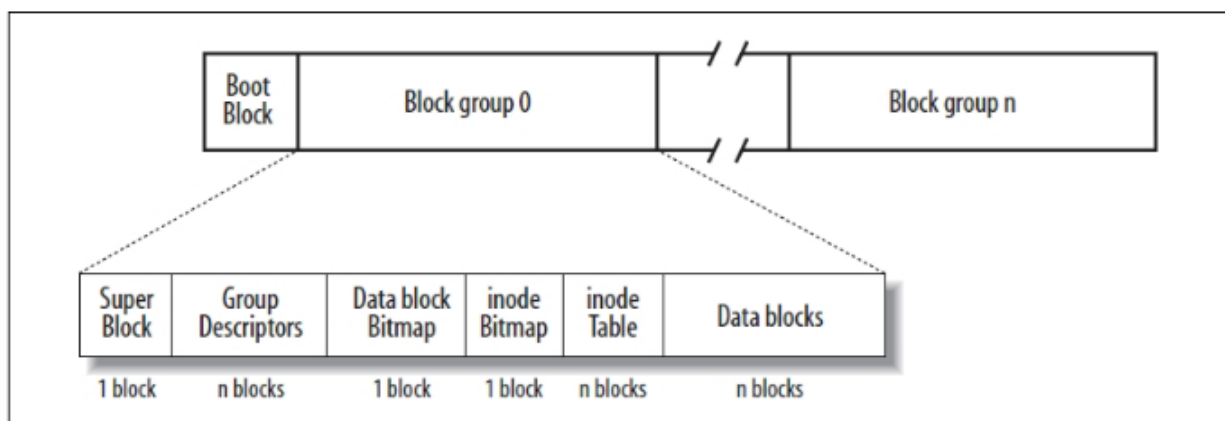
sistema, largo del direntry (para saltar mas facil al siguiente cuando iteramos), largo de nombre, el nombre y el tipo de archivo.

¿Para qué sirven los block groups y los clusters? Motivación para agrupar páginas en bloques, y bloques en grupos de bloques.

Block groups: Permiten organizar el sistema de archivos en unidades más pequeñas y manejables, mejorando la localización de los datos y la eficiencia de acceso. Además, permiten una mejor gestión de la fragmentación y la paralelización de operaciones.

Clusters: Agrupan múltiples bloques de datos para reducir la sobrecarga de gestión y optimizar las operaciones de lectura y escritura, mejorando el rendimiento y la eficiencia.

¿Cuáles son las estructuras más importantes de ext2? Explicar cada una (en particular, hablar del superbloque).



Superblock: Contiene metadatos del sistema de archivos como el tamaño, espacio libre y donde se encuentran los datos. La tabla de inodos, el inode bitmap, etc.

Directory entry: Contiene los datos de un archivo, sea de datos o un directorio. Estos son nombre, tamaño de nombre, número de inodo, tamaño de direntry y tipo de archivo.

Inodo: El más importante de ext2. Contiene metadatos del archivo como permisos, timestamps, tamaño, cantidad de hardlinks, etc. Luego tenemos la sección de bloques directos, bloques indirectos, bloques indirectos dobles, bloques indirectos triples.

Explicar cómo se manejan los bloques libres del disco.

Se gestionan mediante una estructura de datos llamada bitmap. Secuencia de bits donde cada bit representa un bloque de disco.

Permite al fs llevar un registro eficiente de que bloques estan libres para ser asignados a archivos o directorios.

Se almacena al principio o al final de cada block group.

Cada grupo de bloques tiene su bitmap lo que permite realizar la asignacion de bloques de manera eficiente y localizada.

Cuando el sistema de archivos necesita asignar un bloque (por ejemplo, cuando un archivo se está creando o ampliando), se realiza el siguiente proceso:

1. **Buscar un bloque libre:** El sistema de archivos recorre el bitmap de bloques libres para encontrar un bit en **0**, lo que indica que el bloque correspondiente está libre.
2. **Marcar el bloque como ocupado:** Una vez que se encuentra un bloque libre, el sistema de archivos cambia el bit correspondiente a **1**, lo que indica que el bloque está ahora ocupado.
3. **Asignar el bloque:** El bloque se asigna al archivo o directorio que lo necesita. En el caso de un archivo, el bloque se agrega a su inodo, que contiene las direcciones de los bloques de datos.

¿Qué pasa cuando se inicia el sistema luego de haber sido apagado de forma abrupta? Explicar cómo hace el sistema para darse cuenta de si hubo algún error (cuando no hace journaling) y cómo lo arregla. (inconsistencias entre contadores y bitmaps, entre entradas de inodos y bitmaps, entre entradas de directorios y bitmaps)

Sin journaling el sistema de archivos debe llevar a cabo un proceso de recuperación y comprobación de consistencia al reiniciarse para resolver estas inconsistencias.

El sistema de archivos verifica su estructura interna en busca de inconsistencias. Para hacer esto, recurre a varias estructuras de datos esenciales, como los bitmaps de bloques libres, bitmaps de inodos libres, entradas de directorios, los inodos de los archivos y contadores de hardlinks)

Bitmaps de bloques libres: Seguimiento de que bloques estan ocupados y cuales libres. Si el sistema se apaga de manera abrupta el bitmap puede no estar actualizado.

Solucion: Si un bit esta en libre pero en el filesystem aparece como ocupado entonces lo corregimos y viceversa.

Bitmaps de inodos libres: Si un inodo estaba marcado como libre pero se encontraba en uso antes del apagado abrupto -> el bitmap de inodos podria estar inconsistente.

Solucion: Lo mismo, si el inodo es marcado como libre pero esta ocupado lo actualizamos y viceversa. Tengamos en cuenta que estas verificaciones las tiene que hacer el sistema operativo comparando los datos de integridad con lo que realmente es la estructura del fs.

Entradas de directorios: Si el sistema se apaga de manera abrupta mientras se esta escribiendo una entrada de directorio pueden haber inconsistencias entre las entradas de directorio y los inodos asociados.

Solucion: Verificamos que todas las direntrys apuntan a inodos validos y que todos los inodos esten referenciados por entradas de directorios que existan. Si encontramos direntrys que no referencian a ningun inodo, las eliminamos. Lo mismo con inodos, si encontramos inodos que no estan referenciados por ningun direntry, los marcamos como libres.

Contadores de hardlinks: Si un archivo o directorio se elimina pero su contador de enlaces no se actualiza correctamente, puede haber inconsistencias.

Solucion: Revisamos cada inodo y actualizamos los hardlink counters segun las direntrys y otros enlaces asociados. Si encuentra que el contador de enlaces es incorrecto, lo ajusta.

Explicar las diferencias (ventajas y desventajas) entre soft links y hard links. ¿Por qué no podemos usar un hard link para referenciar inodos de otro FS, incluso si está basado en inodos?

Hard link: Es un directory entry que asocia un nombre con un archivo (inodo). Esto implica que cada archivo tiene que tener al menos un inodo.

Soft link: Es un directory entry que apunta a un inodo que en sus bloques de datos tiene la direccion de un archivo. No apunta a otro direntry, si no al path de otro inodo. (Archivo).

Si se elimina el archivo al que hacen referencia...

Hard link: No pasa nada, el archivo sigue existiendo porque hay un hardlink al inodo (solo se eliminan cuando el contador de hardlinks es 0).

Soft link: Se rompe, apunta a algo que ya no existe en el file system.

Los hard links se basan en inodos. Los inodos son una estructura interna específica de un sistema de archivos, y cada sistema de archivos tiene su propio conjunto de inodos,

gestionados de manera independiente.

Cuando creamos un hard link estamos creando un directory entry en el sistema de archivos que apunta al mismo inodo que el direntry original. Si intentamos crear un hardlink en otro sistema de archivos no podemos hacer referencia al mismo inodo ya que el sistema de archivos de destino tienen un conjunto diferente de inodos.

Como los symlinks guardan un path y no un inodo, se pueden vincular a diferentes sistemas de archivos.

Explicar cómo se crean y borran archivos con las estructuras del FS (incluido cómo se modifica el block group). Explicar el caso de borrado en hard links.

Creacion:

1. Se elige un inodo libre del sistema (4 billones es el maximo en ext2) utilizando el bitmap de inodos (en el block group, justo despues del block bitmap) (o sea que hay 2 bitmaps en los block groups. Uno para inodos y otro para bloques, que truco, no? Son las 3 am, tengo frio y sueño). Ponemos ese numero de inodo en 1 (ocupado).
2. Asignamos bloques libres mediante el bitmap de bloques que se encuentra en el block group para almacenar los contenidos del archivo.
3. Modificamos la direntry de la carpeta donde queremos crear el archivo para que apunte a nuestro inodo (el numero de inodo) con su nombre y demas datos (length name, length de dentry, etc). Si el archivo es un directorio, creamos un bloque especial de directorio con su propio inodo.

Borado:

4. Una syscall para deslinkar la dentry del inodo.
5. decrementamos el contador de hardlinks del inodo.
6. Si el contador es 0, liberamos el inodo y los bloques de datos que estaban asociados a este.
7. Se marcan como libre el inodo y los bloques en los bitmaps del block group.